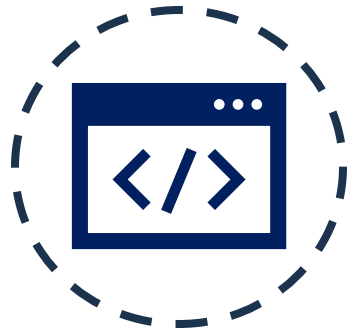




12. Integração *front-end*: React

SCC0219 – Introdução ao Desenvolvimento Web

Prof^a. Dr^a. Bruna C. Rodrigues da Cunha

brunaru@icmc.usp.br

React

- React é uma biblioteca de código aberto para construção de interfaces de usuário (UI) em aplicações web. Desenvolvida pelo Facebook, React permite que os desenvolvedores criem componentes reutilizáveis em JavaScript que podem ser usados para construir interfaces dinâmicas e interativas.

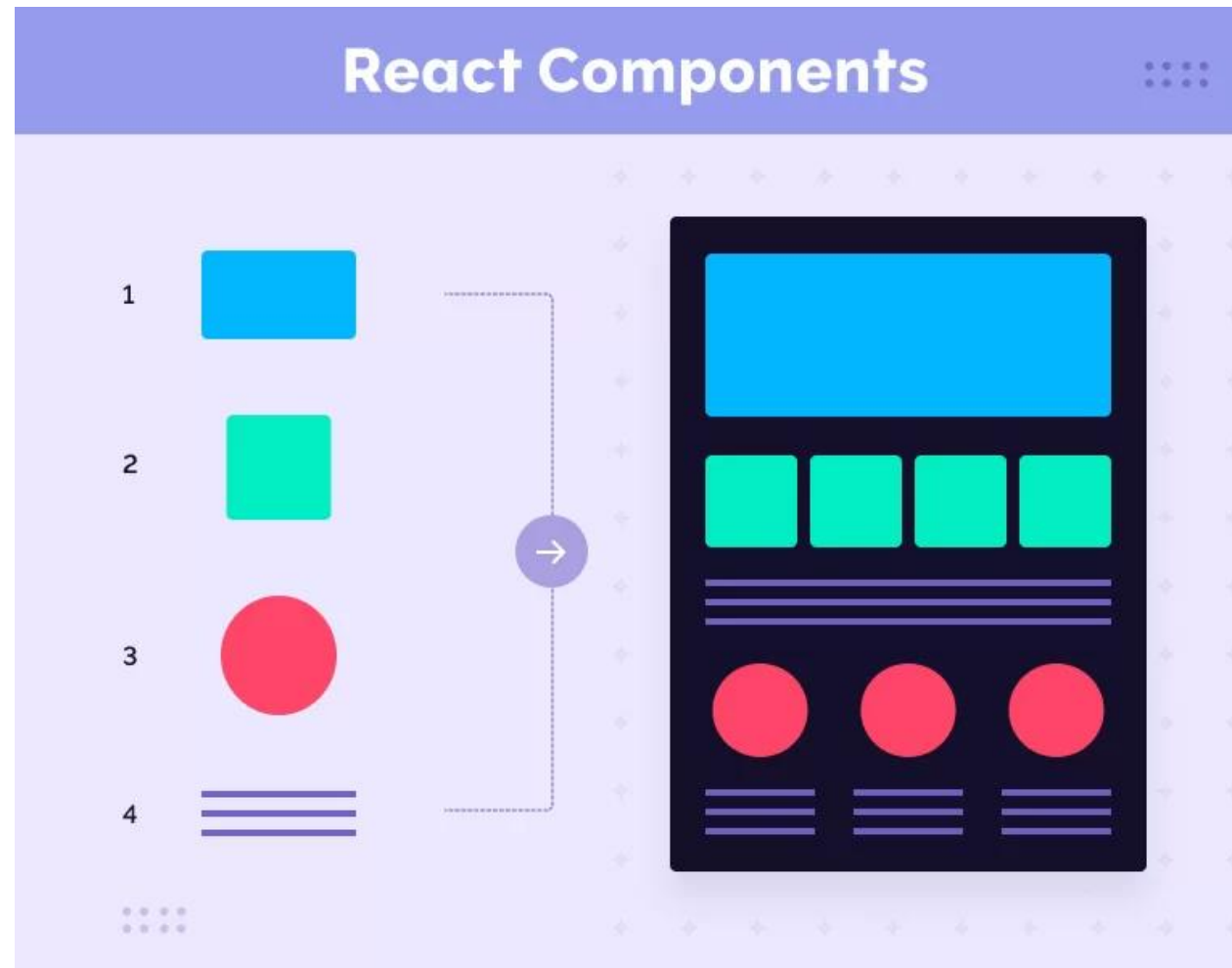


React

- O React utiliza uma abordagem baseada em componentes. Os componentes podem ser compostos de outros componentes menores, permitindo que os desenvolvedores criem interfaces de usuário complexas a partir de partes reutilizáveis e independentes.



React



React

- O React utiliza um conceito chamado de "virtual DOM" para otimizar o desempenho da aplicação.
 - Em vez de atualizar todo o DOM sempre que uma mudança é feita na interface de usuário, o React atualiza somente as partes que foram alteradas, tornando a atualização da interface de usuário muito mais rápida e eficiente.
-

DOM

```
<!DOCTYPE html>
<html>
  <head>
    <title>
      What's DOM
    </title>
  </head>
  <body>
    <h1>DOM</h1>
    <p>DOM is Document Object Model</p>
  </body>
</html>
```

HTML



```
└ DOCTYPE: html
  └ HTML
    └ HEAD
      └ TITLE
        └ #text: What's DOM
    └ BODY
      └ H1
        └ #text: DOM
      └ P
        └ #text: DOM is Document Object Model
```

DOM

Virtual DOM

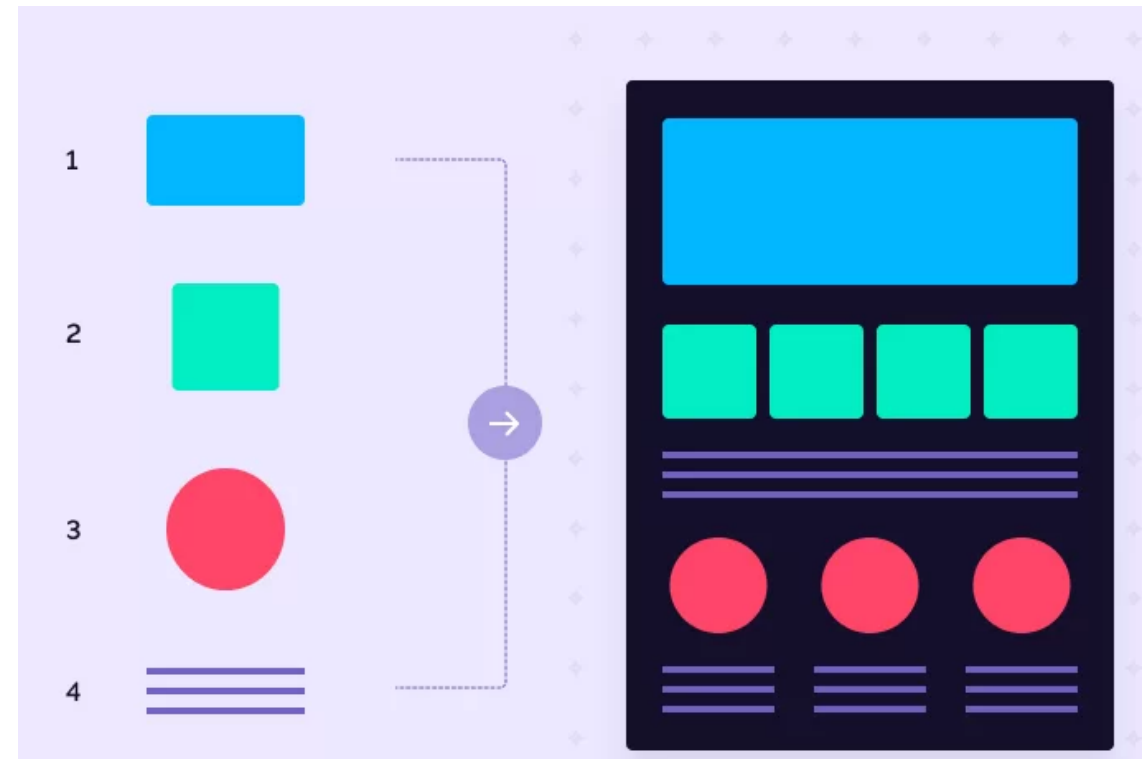
- O Document Object Model (DOM) é representado como uma estrutura de dados em árvore. Após uma alteração, um elemento atualizado e seus filhos precisam ser renderizados novamente para atualizar a interface.
 - Quanto mais descendentes, mais caras serão as atualizações do DOM.
 - Quando uma atualização é feita em um componente React, ele cria uma nova versão do Virtual DOM, que é comparada com a versão anterior para determinar quais elementos do DOM real precisam ser atualizados. Em seguida, o React atualiza apenas os elementos necessários no DOM real, em vez de atualizar toda a árvore de elementos.
-

Componentes

- Componentes são a base da arquitetura do React, que permite que as interfaces de usuário sejam divididas em partes menores e independentes, que podem ser reutilizadas em diferentes partes da aplicação.
 - Cada componente pode ter seu **próprio estado** e **propriedades**, que podem ser passadas para dentro de outro componente a partir do componente pai.
-

Componentes

- Apresentam código reutilizável
- São renderizáveis na tela
- Retornam código declarativo em JSX
- Podem receber propriedades, guardar estados, chamar funções JS e aplicar formatação CSS.



JSX

- JSX é uma extensão da sintaxe do JavaScript que permite escrever código HTML/XML em arquivos JavaScript. Essa sintaxe é usada principalmente com a biblioteca React para definir a estrutura da interface de usuário.
 - Em vez de escrever código HTML separadamente do código JavaScript, **o JSX permite que os desenvolvedores escrevam ambos em um único arquivo**, o que pode tornar o processo de desenvolvimento mais eficiente e intuitivo.
 - JSX não é obrigatório para utilizar React.
-

JSX

- Código JavaScript:

```
const title = React.createElement('h1', {className: 'title'}, 'Hello, world!')
```

- Código JSX:

```
const title = <h1 className="title">Hello, world!</h1>
```

- O código JSX é transformado em JavaScript.
-

Programação Declarativa

- JavaScript é uma linguagem de programação imperativa. Essa abordagem se concentra em descrever passo a passo como o código deve ser executado para produzir um resultado específico.
 - **JSX é considerado uma forma de programação declarativa.**
-

Programação Declarativa

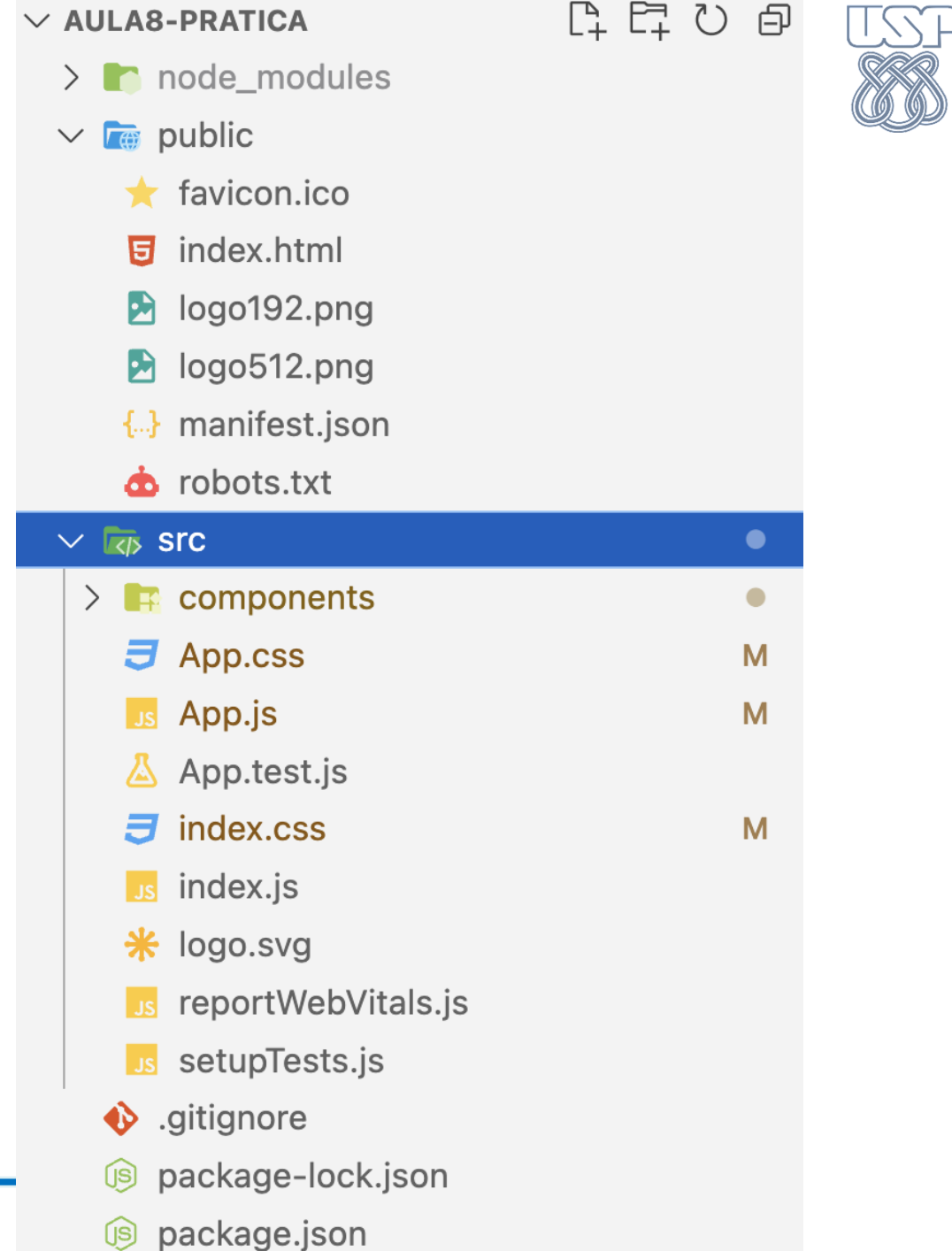
- JSX é considerado uma forma de programação declarativa.
 - Ao usar JSX, o código descreve a aparência e comportamento da interface de usuário com base nos estados e nas propriedades que um componente recebe como entrada.
 - O React se encarrega de atualizar o DOM com base nessa descrição, **sem que o desenvolvedor precise se preocupar com as atualizações no DOM em si.**
 - Em resumo, JSX é considerado uma forma de programação declarativa pois permite que os desenvolvedores descrevam a interface sem se preocupar com a manipulação direta do DOM.
-

Criando um Projeto

- O React é construído em cima do Node.js, então é preciso instalá-lo antes de começar.
- O Vite é uma ferramenta de linha de comando que permite criar projetos React rapidamente, sem a necessidade de configurar manualmente o ambiente de desenvolvimento.
- Uma vez criado o projeto (e após instalar as bibliotecas necessárias), é só executar `npm run dev`.
- Documentação do Vite: <https://vite.dev/>
- Para criar e executar o projeto, utilizar os comandos:

```
npm create vite@latest meu-projeto  
npm i  
npm run dev
```

Estrutura do Projeto



Estrutura do Projeto

- my-app/: é a pasta raiz do projeto.
 - **node_modules/**: é a pasta onde todas as dependências do projeto são armazenadas, incluindo o React e outras bibliotecas que você instalar.
 - **package.json**: é o arquivo de configuração do Node.js que contém informações sobre o projeto, como suas dependências, scripts, autor, licença, entre outros.
 - **public/**: é a pasta de arquivos estáticos, como imagens, fontes e o arquivo index.html, que é o ponto de entrada da aplicação React.
 - **index.html**: é o arquivo HTML que é servido pelo servidor da aplicação. Ele inclui um elemento `<div>` com um id específico (**root**) que é usado pelo React para renderizar a aplicação.
-

Estrutura do Projeto

- `favicon.ico`: é o ícone que é exibido na barra de título do navegador.
 - **src/**: é a pasta que contém o código-fonte da aplicação React.
 - **App.css**: é o arquivo CSS que contém os estilos específicos do componente App.
 - **App.js**: é o componente principal da aplicação, que é renderizado na página inicial.
 - `App.test.js`: é o arquivo de teste do componente App.
 - `index.css`: é o arquivo CSS padrão do projeto.
 - **index.js**: é o arquivo JavaScript que inicializa a aplicação React e renderiza o componente App na página inicial.
-

Componentes React

- Quando trabalhamos com JSX, um componente React deve retornar o código JSX que será renderizado na tela:

```
function MeuComponente() {  
  return (  
    <div>  
      <h1>Olá, mundo!</h1>  
      <p>Este é meu componente React.</p>  
    </div>  
  )  
}  
export default MeuComponente
```

Componentes React

- Podemos usar componentes dentro de outros componentes:

```
import PetLista from './components/PetLista'

function App() {

  return (<div>
    <PetLista />
    </div>)
}
export default App
```

Passando Propriedades (props)

- Componentes podem passar e receber **propriedades**, que geralmente nomeamos como **props**:

```
function Pet(props) {  
  return (<div className="pet">  
    <p>{props.name}</p>  
    <p>{props.type}</p>  
    <p>{props.breed}</p>  
    <p>{props.birth}</p>  
    <p>{props.ClientId}</p>  
    <img src={props.photo} alt={props.name} className="p-img" />  
  </div>)  
}  
export default Pet
```

Passando Propriedades (props)

- Para enviar as propriedades devemos seguir a mesma nomenclatura declarada no componente:

```
function PetLista() {  
  return (<div>  
    <h1>Pets</h1>  
    <Pet  
      name="Shelsiane"  
      type="Dog"  
      breed="Shiba"  
      birth="11/11/2020"  
      ClientId="2"  
      photo="https://upload.wikimedia.org/wikipedia/commons/f/f2/Picography-curious-dog-  
animal-pet-house-home-sm-1.jpg"/>  
    </div>))}
```

CSS

- A aplicação de estilos CSS é simples. Geralmente, cria-se um arquivo CSS com nome igual ao do componente. O componente deve importar o estilo. A aplicação de classes no JSX é feita pelo atributo className.

```
import “./PetLista.css”
```

```
function PetLista() {  
  return (<div className='p-basico'>  
    <h1>Pets</h1>  
    <Pets name=“Shelsiane” ... ClientId=“2”/>  
  </div>))}
```

Listando Vários

- Em React, a listagem de elementos em tela geralmente é feita utilizando o método `.map()`, que é uma função nativa de arrays no JavaScript.
 - Esse recurso permite percorrer cada item de uma lista e retornar um novo elemento para cada um deles, resultando em uma renderização dinâmica e eficiente.
 - Na prática, em vez de escrever manualmente vários componentes iguais, o `.map()` automatiza a criação desses elementos com base nos dados do array, tornando o código mais limpo, reutilizável e fácil de manter.
 - Além disso, cada item renderizado deve conter uma prop `key` única, para que o React consiga identificar e gerenciar corretamente cada elemento na interface.
-

Listando Vários

```
const pets = [  
  { name: "Shelsiane", type: "Dog", breed: "Shiba", birth: "11/11/2020", ClientId: "2", photo:  
    "https://upload.wikimedia.org/wikipedia/commons/f/f2/Picography-curious-dog-animal-pet-house-home-sm-1.jpg" },  
  { name: "Mimi", type: "Cat", breed: "Siamês", birth: "05/06/2019", ClientId: "4", photo:  
    "https://upload.wikimedia.org/wikipedia/commons/7/7e/Siam_lilacpoint.jpg" },  
  { name: "Thor", type: "Dog", breed: "Golden Retriever", birth: "23/03/2021", ClientId: "7", photo:  
    "https://upload.wikimedia.org/wikipedia/commons/9/99/Golden_Retriever_Carlos_%2810540995976%29.jpg" }  
]
```


Listando Vários

- Para renderizar vários componentes com propriedades distintas, podemos percorrer os dados (via função map) e retornar um componente para cada:

```
function PetLista() {  
  return (<div>  
    {pet.map(function (pet, index) {  
      return (<Pet  
        key={index}  
        name={pet.name}  
        type={pet.type}  
        birth={pet.birth}  
        ClientId={pet.ClientId}  
        photo={pet.photo} />)  
      })}  
    </div>))}
```

Props de evento

- No React, a propriedade de evento (ou “event prop”) é a forma como um componente recebe uma função para reagir a interações do usuário, como cliques, digitação ou envio de formulários.
- Quando uma função é passada como valor de uma prop de evento – por exemplo, onClick ou onChange – o componente armazena a referência e a chama automaticamente quando a interação ocorre.
- Essa abordagem torna os componentes mais reutilizáveis e desacoplados, já que a lógica de comportamento fica externa ao componente que emite o evento.

Props de evento

```
export default function App() {  
  
  function handleClick() {  
    console.log("O botão foi clicado!");  
  }  
  
  return (  
    <div>  
      <button onClick={handleClick}>Clique aqui</button>  
    </div>  
  );  
}
```

State

- O **state** no React é um objeto JavaScript que representa o estado interno de um componente.
- O state é utilizado para armazenar dados que podem mudar ao longo do tempo e que afetam a renderização do componente.
- Quando o state de um componente é atualizado, o React automaticamente (re)renderiza o componente para refletir as mudanças.
- Isso significa que o state é uma das maneiras de fazer um componente React ser dinâmico.
- O **useState** é um *hook* do React, que permite que um componente funcional tenha estado interno. É a maneira mais simples de gerenciar o state.

```
const [state, setState] = useState(initialState)
```

State

```
import { useState } from 'react'

function MeuComponente() {
  let [contador, setContador] = useState(0)

  function clicou() {
    setContador(contador + 1)
  }

  return (<div>
    <p>Contador: {contador}</p>
    <button onClick={clicou}>Clique aqui</button>
  </div>))
export default MeuComponente
```

Condicional

- Em JSX, é possível usar condicionais para renderizar elementos de forma condicional. Existem algumas formas de fazer isso, mas uma das mais comuns é usando a expressão ternária:

```
function MeuComponente(props) {  
  return (<div>  
    {props.mostrarTexto ? <p>Texto em caso  
positivo!</p> : <p>Texto em caso negativo.</p>}  
    </div>)  
}
```

Condicional

```
import { useState } from "react"
export default function App() {
  const [foiClicado, setFoiClicado] = useState(false)
  function handleClick() {
    console.log("O botão foi clicado!")
    setFoiClicado(true)
  }

  return (
    <div>
      <button onClick={handleClick}>Clique aqui</button>
      {foiClicado && <p>O botão foi pressionado</p>}
    </div>
  )
}
```

Formulário

- Um formulário deve ter funções associadas para seus campos e submissão de dados inseridos. Pode ser chamada uma função do componente pai para tratar os dados.

```
import ClientForm from “./ClientForm”
function Cadastrar() {
  function atualizar(nome) {
    // faz o envio de dados para um web service...
  }
  return (<div>
    <ClientForm onCadastrar={atualizar} />
  </div>))
```

Formulário

```
import { useState } from 'react'

function NomeForm(props) {
  let [nome, setNome] = useState('')

  function onNomeChange(event) {
    setNome(event.target.value)
  }

  function onEnviar(event) {
    event.preventDefault()
    props.onCadastrar(nome)
    setNome('')
  }

  return (<div>
    <form onSubmit={onEnviar}>
      <label>Nome: </label> <input type="text" value={nome} onChange={onNomeChange} />
      <button type='submit'>Cadastrar</button>
    </form></div>)}
  export default NomeForm
```

Atividade

- Crie uma aplicação React simples que exiba o nome de um usuário e o número de seguidores que ele tem em uma rede social.
 - O nome do usuário e o número de seguidores devem ser passados como propriedades para um componente.
 - O componente deve exibir essas informações em elementos `h1` e `p`, respectivamente.
 - Além disso, o componente deve ter um botão que, ao ser clicado, aumenta o número de seguidores em 1.
 - O número de seguidores atualizado deve ser exibido imediatamente após o botão ser clicado, sem a necessidade de recarregar a página.
 - Você deve usar o `useState` para gerenciar o estado do número de seguidores.
 - Fazer um CSS que formate o componentes.
-